Анализ IT-вакансий разработчиков C и C++

RustFS, собственный MapReduce и DuckDB
Вариант 6

Выполнил: Волков Владимир
Дата экспериментов: 21 октября 2026 года

Цель работы

Подготовил данные вакансий, загрузил их в RustFS и реализовал два независимых способа подсчёта навыков и их сочетаний. Проверил совпадение результатов MapReduce и DuckDB, измерил время и пик памяти. Исследовал связь сочетаний навыков с зарплатой и сравнил три региона.

1. Данные и подготовка

Для исследования выбрал набор **IT Vacancies from HeadHunter Website**, опубликованный на Kaggle пользователем ilyazawilsiv. Проанализировал исторический срез вакансий с датами публикации 18.09.2023-17.10.2023.

Из исходной таблицы сохранил необходимые столбцы всех вакансий. Списки навыков преобразовал в JSON-массивы. Использовал готовое поле key_skills; навыки из описаний не извлекал, отсутствующие зарплаты не заполнял.

Показатель	Значение
Строк в подготовленном объекте	68 770
Размер JSONL	30 732 626 байт (29,3 MiB)
Вакансий после отбора варианта	869
С навыками / без навыков	676 / 193

Подготовил полный объект из 68 770 строк. Отбор варианта выполнял независимо в MapReduce и DuckDB при чтении данных из S3.

2. Объектное хранилище RustFS

Для хранения данных использовал RustFS в Docker Desktop на macOS. Программы обращались к S3 API через порт **9000**, состояние бакета проверял в веб-консоли на порту **9001**.

Параметр	Фактическое значение
S3 endpoint	http://127.0.0.1:9000
Бакет	hh-lake
Входной объект	raw/it_vacancies.jsonl

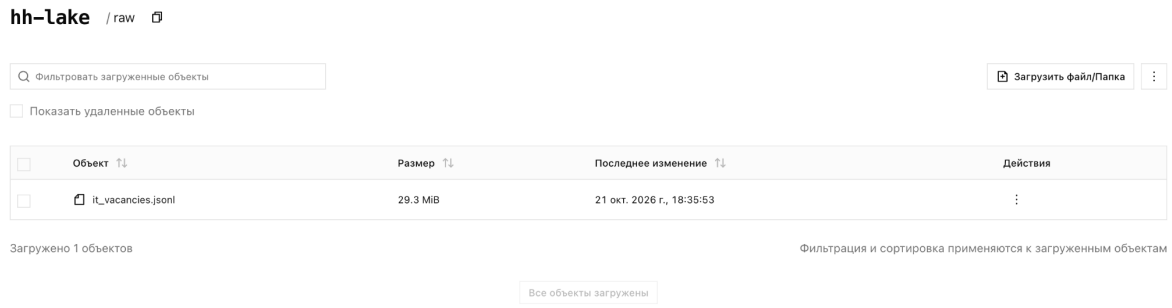


Рисунок 1. Входной объект в бакете hh-lake. В консоли видны имя, размер и время загрузки.



Рисунок 2. Вывод docker compose ps: контейнер RustFS запущен, порты 9000 и 9001 опубликованы.

Загрузил JSONL в hh-lake и сохранил протокол в evidence/storage.json. Проверил совпадение SHA-256 подготовленного файла и метаданных объекта:

```
d9bb991e4932c95d2231decb70f21b31338c
4be0a3b89cf9d1a8cbfefd20e2b5
```

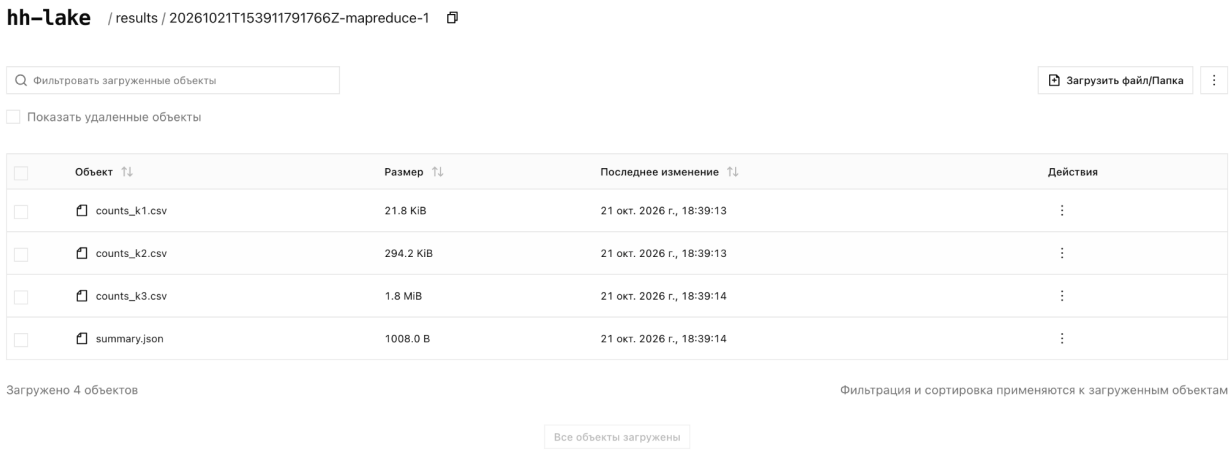


Рисунок 3. Результаты успешного MapReduce в results/: отдельные файлы counts_k1.csv, counts_k2.csv, counts_k3.csv и сводка summary.json.

Для каждого запуска сохранил частоты и сводку локально и выгрузил файлы в hh-lake/results/.

3. Отбор варианта и собственный MapReduce

Условие варианта

Для отбора использовал professional_roles_id = 96 («Программист, разработчик») и явное упоминание C или C++ в названии. Учёл кириллическую «С», обозначение «Си» и с developer. Исключил ложные совпадения с 1С, C#, Objective-C и инициалами.

Условие реализовал отдельно в Python и SQL. В выборку включил явно обозначенные C/C++-вакансии. Общие названия без указания языка могли быть пропущены. SQL-условие привёл в приложении.

Подготовка навыков внутри вакансии

Привёл названия навыков к нижнему регистру, удалил пробелы по краям и повторы внутри вакансии, отсортировал списки. Разные исходные теги, например c++ и c/c++, сохранил отдельно.

Алгоритм расчёта

1. **Чтение.** Реализовал получение объекта через boto3.get_object и потоковое чтение StreamingBody.iter_lines с буфером 64 KiB. Формировал чанки по 500 строк непосредственно из S3.

2. **Map.** Запустил два рабочих процесса. В каждом отбирал вакансии и формировал сочетания из одного, двух и трёх навыков через itertools.combinations. Перестановки навыков не учитывал как новые сочетания.

3. **Reduce.** Суммировал результаты успешных задач в главном процессе. Ограничил очередь четырьмя незавершёнными задачами, чтобы не накапливать весь входной объект в памяти.

```
body = s3.get_object(Bucket=bucket, Key=key)["Body"]
for line in body.iter_lines(chunk_size=64 * 1024):
    # строки объединяются в чанки по 500
    ...
for k in (1, 2, 3):
    counts[k].update(combinations(skills, k))
```

Чтение, обработку чанков и объединение частот реализовал в функциях chunks_from_s3, mapper и mapreduce файла lab.py.

Оптимизация

Реализовал наивный вариант с передачей каждого вхождения и оптимизированный вариант с локальным Counter (combiner). Предварительное объединение одинаковых ключей уменьшает объём данных, передаваемых между процессами.

4. DuckDB и проверка совпадения

В DuckDB настроил чтение того же объекта s3://hh-lake/raw/it_vacancies.jsonl через httpfs. Отбор и нормализацию выполнил запросом sql/variant.sql. Python-функцию отбора и частоты MapReduce в SQL не использовал.

Развернул навыки в строки с номером позиции. Для пар задал i < j, для троек i < j < k, исключив перестановки и повтор одного навыка. Подсчёт реализовал следующими запросами из sql/counts.sql:

```
CREATE TEMP TABLE units AS
SELECT id, i, skills[i] AS skill
FROM selected, range(1, len(skills) + 1) AS positions(i);

CREATE TEMP TABLE counts1 AS
SELECT [skill] AS skills, count(*)::BIGINT AS n
FROM units GROUP BY skills;

CREATE TEMP TABLE counts2 AS
SELECT [a.skill, b.skill] AS skills, count(*)::BIGINT AS n
FROM units a JOIN units b ON a.id=b.id AND a.i<b.i
GROUP BY skills;

CREATE TEMP TABLE counts3 AS
SELECT [a.skill, b.skill, c.skill] AS skills,
       count(*)::BIGINT AS n
FROM units a
JOIN units b ON a.id=b.id AND a.i<b.i
JOIN units c ON a.id=c.id AND b.i<c.i
GROUP BY skills;
```

Размер сочетания	Различных ключей	Сумма частот
1 навык	747	3 833
2 навыка	6 800	13 468
3 навыка	31 306	43 504

Результат проверки. Сравнил все ключи и частоты для k = 1, 2, 3, число вакансий, хеш выбранных ID и распределение размеров стеков. Во всех девяти запусках получил полное совпадение: full equality OK.

Почему пар и троек больше

Рост числа сочетаний объяснил вкладом каждой вакансии: для m навыков получаются m одиночных вхождений, m(m−1)/2 пар и m(m−1)(m−2)/6 троек. При 10 навыках это 10, 45 и 120. Проверил суммы вкладов по всей выборке. Повторяющиеся сочетания увеличивают частоту уже существующего ключа.

5. Время, память и результат оптимизации

Провёл эксперименты на macOS 15.8.1, arm64, Python 3.12.14. Использовал 2 процесса MapReduce, чанки по 500 строк и 2 потока DuckDB. Выполнил по три попытки каждого метода, чередуя методы и запуская каждую попытку в отдельном процессе.

Измерял полное время запуска Python, чтения S3, расчёта, сортировки, записи и выгрузки результатов. httpfs установил заранее. Пик памяти оценивал по сумме RSS процесса и живых потомков с опросом каждые 10 мс. Память RustFS не включал. Общие страницы могли учитываться повторно, короткие пики - быть пропущены.

Метод	Попытка	Время, с	Пик RSS, MiB
Наивный MapReduce	1	0.920	173.2
MapReduce + combiner	1	0.854	174.0
DuckDB	1	0.946	294.9
Наивный MapReduce	2	0.837	170.0
MapReduce + combiner	2	0.876	172.1
DuckDB	2	0.706	289.3
Наивный MapReduce	3	0.889	172.3
MapReduce + combiner	3	0.849	171.0
DuckDB	3	0.662	286.7

Медианы трёх запусков

Метод	Время, с	Пик RSS, MiB
Наивный MapReduce	0.889	172.3
MapReduce + combiner	0.854	172.1
DuckDB	0.706	289.3

После добавления combiner получил снижение медианы времени на **3,9%** и медианы пика RSS примерно на **0,1%**. Небольшой выигрыш связал с размером выборки и накладными расходами чтения S3, запуска процессов и обмена данными. По трём попыткам устойчивость ускорения не устанавливал.

В этих условиях DuckDB показал наименьшую медиану времени, но медиана пика RSS оказалась в 1,68 раза выше, чем у MapReduce с combiner. Кэш между попытками не очищал; сравнивал полный локальный пайплайн.

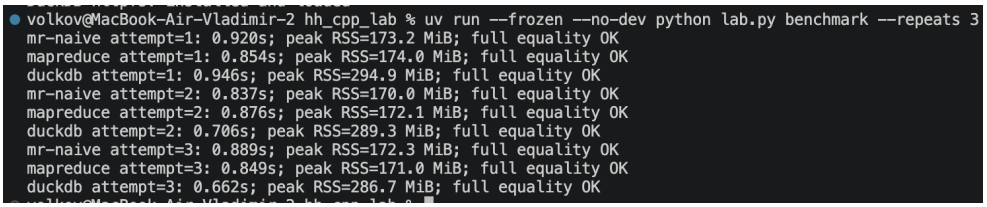


Рисунок 4. Реальный вывод всех девяти запусков: время, пик памяти и full equality OK. В таблицах сохранена точность вывода терминала.

6. Зарплаты: метод сравнения и пары навыков

Для сравнения взял положительные зарплаты в рублях с salary_gross = false («на руки»). Для двух границ вычислил середину вилки, для одной использовал указанное значение. Некорректные вилки, другие валюты, зарплаты до налогов и пропуски исключил.

Базой пары выбрал **более частый отдельный навык**, базой тройки - **самую частую составляющую пару**. Частоту определял по всем вакансиям варианта, при равенстве использовал лексикографический порядок. Базовая выборка включает вакансии с дополнительным навыком.

Рассчитал разницу медиан сочетания и базы. При пороге не менее пяти зарплат получил 555 сочетаний: 145 пар и 410 троек. Для каждого размера выбрал по три сочетания с наибольшей положительной и отрицательной разницей в рублях; одинаковые значения разницы сохранял как равные результаты.

Пары с наибольшей положительной разницей

Сочетание	Медиана, руб.	База	Медиана базы, руб.	Разница, руб.
ms visual studio + multithread programming	325 000 n = 5	ms visual studio частота 24	175 000 n = 12	+150 000
c++ + multithread programming	325 000 n = 6	c++ частота 436	180 000 n = 81	+145 000
ms visual studio + ооп	325 000 n = 6	ооп частота 87	200 000 n = 36	+125 000

Пары с наибольшей отрицательной разницей

Сочетание	Медиана, руб.	База	Медиана базы, руб.	Разница, руб.
c# + c++	100 000 n = 10	c++ частота 436	180 000 n = 81	-80 000
c++ + sql	100 000 n = 8	c++ частота 436	180 000 n = 81	-80 000
tcp/ip + ооп	135 000 n = 5	ооп частота 87	200 000 n = 36	-65 000

Наблюдение. Для MS Visual Studio + multithread programming получил медиану 325 000 руб. против 175 000 руб. у базы: +150 000 руб. (+85,7%). В сочетании 4 из 5 зарплатных вакансий требуют опыта 3-6 лет, у базы - 8 из 12. Предположил, что состав по опыту объясняет часть разницы; причинный эффект не оценивал.

Для C# + C++ и C++ + SQL выявил медиану ниже базы C++ на 80 000 руб. (−44,4%). Доля вакансий с опытом 3-6 лет у C# + C++ равна 50%, у базы - 66,7%. Связал наблюдение с возможными различиями уровня позиций, а не с отрицательным влиянием самого навыка.

7. Тройки навыков и интерпретация результатов

Тройки с наибольшей положительной разницей

Сочетание	Медиана, руб.	База	Медиана базы, руб.	Разница, руб.
c++ + ms visual studio + multithread programming	325 000 n = 5	c++ + ms visual studio частота 17	225 000 n = 9	+100 000
c++ + ms visual studio + ооп	325 000 n = 5	c++ + ооп частота 62	225 000 n = 27	+100 000
c++ + multithread programming + ооп	325 000 n = 5	c++ + ооп частота 62	225 000 n = 27	+100 000

Тройки с наибольшей отрицательной разницей

Сочетание	Медиана, руб.	База	Медиана базы, руб.	Разница, руб.
boost + cmake + ооп	200 000 n = 7	boost + ооп частота 20	250 000 n = 15	-50 000
boost + design patterns + ооп	200 000 n = 7	boost + ооп частота 20	250 000 n = 15	-50 000
boost + git + stl	200 000 n = 5	boost + stl частота 60	250 000 n = 17	-50 000

Для C++ + MS Visual Studio + multithread programming получил медиану 325 000 руб., на 100 000 руб. (+44,4%) выше базы C++ + MS Visual Studio. Сравнил 5 зарплат тройки с 9 зарплатами базы. Многопоточность встретилась среди лидеров пар и троек, однако эти подвыборки пересекаются.

Для Boost + CMake + ООП получил разницу −50 000 руб. (−20%) относительно Boost + ООП. Все 7 зарплатных вакансий тройки и все 15 вакансий базы требуют опыта 3-6 лет. Поэтому различие не объясняется только этой категорией опыта; возможны различия работодателей, задач и городов.

Дополнительная проверка по одиночному навыку

Пример тройки	Самый частый навык	Его частота	Медиана навыка, руб.
c++ + ms visual studio + multithread programming	c++	436	180 000 (n = 81)
boost + cmake + ооп	ооп	87	200 000 (n = 36)

Дополнительно сравнил тройки с самым частым одиночным навыком; его показатели сохранил в salary_effects.csv. Основной базой тройки оставил пару, чтобы оценить добавление одного навыка.

Вывод. Выявил положительные и отрицательные различия медиан для сочетаний навыков. Интерпретировал их как связь с составом вакансий. Причинный эффект навыков и статистическую значимость различий не оценивал; результаты малых подвыборок требуют осторожной интерпретации.

8. Сравнение трёх регионов

Сравнил Москву, Санкт-Петербург и Новосибирск. Доли навыков рассчитывал от всех вакансий варианта в городе, включая записи без навыков. Для зарплат использовал ту же выборку net RUR.

Город	Вакансий	С навыками	n зарплат	Медиана, руб.
Москва	398	309	57	200 000
Санкт-Петербург	186	149	46	137 500
Новосибирск	28	20	9	150 000

В Москве получил 398 вакансий: в 2,14 раза больше, чем в Санкт-Петербурге, и в 14,2 раза больше, чем в Новосибирске. Московская медиана выше петербургской на 62 500 руб. (45,5%). Зарплаты по выбранной политике доступны для 14,3%, 24,7% и 32,1% вакансий этих городов соответственно.

Для Новосибирска получил медиану 150 000 руб. против 137 500 руб. в Санкт-Петербурге. Учёл, что новосибирская оценка основана только на девяти зарплатах и не характеризует весь рынок города.

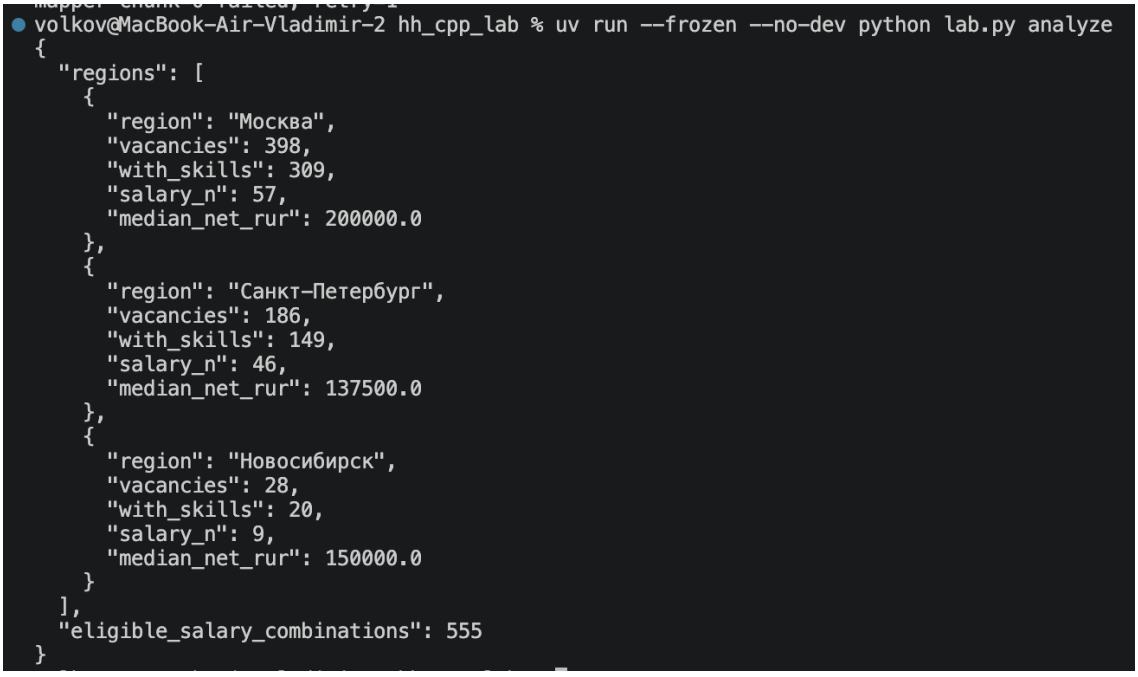


Рисунок 5. Вывод analyze: сводка по трём городам и 555 сочетаний, прошедших зарплатный порог.

Для каждого города выделил пять самых частых навыков, пар и троек. В таблицах n обозначает число зарплатных наблюдений; при их отсутствии указал «нет данных».

9. Региональные навыки: Москва

Топ-5 для каждого размера сочетания

k	Навык или сочетание	Частота	Доля, %	n	Медиана, руб.
1	c++	196	49.2	29	200 000
1	linux	166	41.7	22	207 500
1	c/c++	92	23.1	17	200 000
1	git	85	21.4	10	200 000
1	qt	65	16.3	11	210 000
2	c++ + linux	96	24.1	12	205 000
2	c/c++ + linux	63	15.8	12	205 000
2	git + linux	63	15.8	10	200 000
2	c++ + git	54	13.6	7	200 000
2	c++ + qt	54	13.6	11	210 000
3	c++ + git + linux	38	9.5	7	200 000
3	c++ + linux + qt	26	6.5	4	205 000
3	c++ + linux + python	24	6.0	3	275 000
3	c++ + git + qt	21	5.3	2	165 000
3	c++ + c/c++ + linux	20	5.0	5	200 000

k = 1 - отдельные навыки; k = 2 - пары; k = 3 - тройки. Доля = частота / все вакансии варианта в этом городе × 100%. Группы сочетаний пересекаются, поэтому доли не суммируются до 100%.

Наблюдения по городу

C++ встречается в 49,2% всех вакансий города, Linux - в 41,7%. Наиболее частая пара C++ + Linux имеет 96 вакансий (24,1%), а тройка C++ + Git + Linux - 38 (9,5%). Это показывает, что C++ в данном сегменте часто соседствует с Linux и Git.

У пары C++ + Linux медиана 205 000 руб. при n = 12, у тройки C++ + Git + Linux - 200 000 руб. при n = 7. Само добавление Git здесь не связано с более высокой медианой. Для тройки C++ + Linux + Python медиана 275 000 руб., но наблюдений только три; по этой цифре нельзя выбирать наиболее выгодный стек.

9. Региональные навыки: Санкт-Петербург

Топ-5 для каждого размера сочетания

k	Навык или сочетание	Частота	Доля, %	n	Медиана, руб.
1	c++	89	47.8	16	145 000
1	linux	80	43.0	25	145 000
1	c/c++	58	31.2	19	140 000
1	git	42	22.6	16	130 000
1	qt	32	17.2	16	125 000
2	c/c++ + linux	43	23.1	16	142 500
2	c++ + linux	41	22.0	9	150 000
2	git + linux	29	15.6	12	122 500
2	linux + qt	24	12.9	14	132 500
2	c++ + git	22	11.8	7	130 000
3	c/c++ + git + linux	16	8.6	8	122 500
3	c/c++ + linux + qt	15	8.1	10	132 500
3	c++ + git + linux	14	7.5	4	125 000
3	c++ + linux + qt	13	7.0	6	125 000
3	c++ + c/c++ + linux	11	5.9	3	140 000

k = 1 - отдельные навыки; k = 2 - пары; k = 3 - тройки. Доля = частота / все вакансии варианта в этом городе × 100%. Группы сочетаний пересекаются, поэтому доли не суммируются до 100%.

Наблюдения по городу

Пять ведущих одиночных тегов те же, что в Москве: C++, Linux, C/C++, Git и Qt. Однако тег C/C++ здесь встречается в 31,2% вакансий против 23,1% в Москве. Ведущая пара - C/C++ + Linux (43 вакансии), тогда как в Москве лидирует C++ + Linux.

Пара C++ + Linux имеет медиану 150 000 руб. при n = 9; в Москве для неё 205 000 руб. при n = 12. Для C/C++ + Linux + Qt в Санкт-Петербурге есть 10 зарплатных наблюдений и медиана 132 500 руб. Сравнение подтверждает различия конкретных подвыборок, но не отделяет город от состава работодателей и уровня позиций.

9. Региональные навыки: Новосибирск

Топ-5 для каждого размера сочетания

k	Навык или сочетание	Частота	Доля, %	n	Медиана, руб.
1	c++	14	50.0	3	80 000
1	linux	13	46.4	5	150 000
1	c/c++	7	25.0	2	160 000
1	git	6	21.4	2	175 000
1	python	5	17.9	0	нет данных
2	c++ + linux	8	28.6	2	115 000
2	c/c++ + linux	7	25.0	2	160 000
2	git + linux	6	21.4	2	175 000
2	c++ + c/c++	4	14.3	0	нет данных
2	c++ + git	4	14.3	1	150 000
3	c++ + c/c++ + linux	4	14.3	0	нет данных
3	c++ + git + linux	4	14.3	1	150 000
3	c/c++ + git + linux	4	14.3	1	200 000
3	bash + linux + python	3	10.7	0	нет данных
3	c + git + linux	3	10.7	2	175 000

k = 1 - отдельные навыки; k = 2 - пары; k = 3 - тройки. Доля = частота / все вакансии варианта в этом городе × 100%. Группы сочетаний пересекаются, поэтому доли не суммируются до 100%.

Наблюдения по городу

C++ отмечен в 50,0% вакансий, Linux - в 46,4%; доля пары C++ + Linux составляет 28,6%, что выше московских 24,1% и петербургских 22,0%. При этом абсолютное число таких вакансий равно только восьми: высокая доля не означает большой рынок.

Python входит в пятёрку одиночных навыков (5 вакансий), однако ни одна из этих вакансий не имеет зарплаты, подходящей под выбранную политику. У C++ медиана 80 000 руб. рассчитана всего по трём вакансиям, а у Git - 175 000 руб. по двум. Эти числа не доказывают зарплатное преимущество Git над C++.

10. Проверка повторного выполнения маппера

Провёл эксперимент со сбоем задачи маппера. Для чанка 0 вызвал исключение после map, до добавления частот в reducer. Реализовал один повтор и добавление промежуточных результатов только после успеха, исключив двойной учёт чанка.

```
uv run --frozen --no-dev python lab.py failure-test
Mapper failure, retry and full equality: OK
```

```
volkov@MacBook-Air-Vladimir-2 hh_cpp_lab % head -n 1 results/failure-*/stdout.log
mapper chunk=0 failed; retry=1
```

Рисунок 6. В stdout.log зафиксирован сбой чанка 0 и один повтор: mapper chunk=0 failed; retry=1.


```
volkov@MacBook-Air-Vladimir-2 hh_cpp_lab % cat evidence/mapper_failure.json
{
  "equal": true,
  "left": "20261021T153911791766Z-mr-naive-1",
  "right": "failure-1792597267732800000",
  "selected_rows": 869,
  "source": {
    "endpoint": "http://127.0.0.1:9000",
    "bucket": "hh-lake",
    "key": "raw/it_vacancies.jsonl",
    "etag": "\"1a60b55a389a2caceebfca02724c6847-4\"",
    "bytes": 30732626,
    "sha256": "d9bb991e4932c95d2231decb70f21b31338c4be0a3b89cf9d1a8cbfefd20e2b5"
  },
  "full_comparison_k": [
    1,
    2,
    3
  ],
  "retries": 1,
  "metrics": {
    "seconds": 0.8743714170414023,
    "peak_rss_mib": 171.21875,
```


Рисунок 7. Протокол проверки: equal = true, 869 вакансий, сравнение всех k = 1, 2, 3 и retries = 1.

После повтора получил полное совпадение с эталонным успешным запуском. Протокол сохранил в evidence/mapper_failure.json, журнал - в results/failure-…/stdout.log.

Эксперимент подтвердил корректность повторного выполнения задачи при исключении. Отказы процесса, компьютера и сети в этом эксперименте не моделировал.

11. Выводы

Отобрал 869 вакансий варианта 6 и получил одинаковые частоты 747 навыков, 6 800 пар и 31 306 троек двумя независимыми способами. Проверил совпадение во всех девяти запусках и после сбоя задачи. Результаты для всех трёх размеров сочетаний сохранил в RustFS.

Установил, что в измеренном пайплайне DuckDB быстрее по медиане времени (0,706 с), а MapReduce с combiner использует меньше памяти (172,1 MiB против 289,3 MiB). После оптимизации MapReduce получил снижение медианы времени примерно на 3,9% при почти неизменной памяти.

Выявил наиболее высокие положительные различия медиан для сочетаний с multithread programming. Среди отрицательных примеров выделил C# + C++, C++ + SQL и Boost + CMake + ООП. По регионам установил сходство ведущих навыков при разных объёмах выборок и зарплатных медианах.

Последовательность выполненных команд

```
uv run --frozen --no-dev python lab.py prepare
uv run --frozen --no-dev python lab.py upload
uv run --frozen --no-dev python lab.py setup-duckdb
uv run --frozen --no-dev python lab.py benchmark --repeats 3
uv run --frozen --no-dev python lab.py failure-test
uv run --frozen --no-dev python lab.py analyze
```

Исходный код и результаты

Содержимое	Путь в проекте
Python-код и независимый SQL	lab.py; source_tools.py; sql/
Конфигурация хранилища	infra/docker-compose.yml
Протокол загрузки, проверки, отказа	evidence/storage.json; verification.json; mapper_failure.json
Профилирование и аналитика	results/benchmark.json; analysis.json; salary_effects.csv; regions_summary.csv; regions_skills.csv
Полные частоты по запускам	results/<имя запуска>/counts_k1.csv, counts_k2.csv, counts_k3.csv
Скриншоты доказательств	evidence/screenshots/

Источник данных

Kaggle: IT Vacancies from HeadHunter Website (ilyazawilsiv). Полный адрес и происхождение подготовленного CSV сохранены в data/provenance.json.

Приложение. Независимый SQL-отбор варианта

Для DuckDB написал запрос отбора варианта из sql/variant.sql. Таблицу raw получил чтением JSONL из S3. Убрал из названия ложные маркеры 1C, Objective-C и C#, затем применил условия языка без вызова функций Python.

```
CREATE TEMP TABLE selected AS
WITH cleaned AS (
    SELECT *, regexp_replace(
      regexp_replace(
        regexp_replace(name,
          '(^|[^\p{L}\p{N}_])1[\s-]*[cc]($|[^\p{L}\p{N}_])', '\1 \2', 'gi'),
        'objective[\s-]*[cc]', '', 'gi'),
      '[CC]\s*#', '', 'g') AS clean_name
    FROM raw
)
SELECT * EXCLUDE (key_skills, clean_name),
       list_sort(list_distinct(list_filter(
         list_transform(key_skills, x -> lower(trim(x))), x -> x <> ''
       ))) AS skills
FROM cleaned
WHERE professional_roles_id = '96' AND (
    regexp_matches(name, '(^|[^\p{L}\p{N}_])[ccCC]\+\+($|[^+])')
 OR regexp_matches(clean_name, '(^|[^\p{L}\p{N}_.])[CC]($|[^\p{L}\p{N}_.#+])')
 OR regexp_matches(clean_name, '(^|[^\p{L}\p{N}_])си($|[^\p{L}\p{N}_])', 'i')
 OR regexp_matches(clean_name, '(^|[^\p{L}\p{N}_])c\s+(developer|programmer)($|[^\p{L}\p{N}_])')
);
```

Использовал Unicode-классы букв и цифр для проверки границ слов и исключения ложных совпадений. При верификации сравнил ID выбранных вакансий, чтобы проверить состав выборок, а не только их размер.

Как связаны реализации и контрольные результаты

Этап	Python / MapReduce	DuckDB
Чтение	boto3, StreamingBody	read_json, httpfs, S3
Отбор	source_tools.is_variant	SQL WHERE из приложения
Сочетания	itertools.combinations	соединения позиций i < j < k
Агрегация	Counter, reducer	GROUP BY, count(*)
Результат	CSV для k = 1, 2, 3	CSV для k = 1, 2, 3

При одинаковых исходном объекте и нормализации независимо реализовал отбор и подсчёт в Python и SQL. Совпадение полных результатов использовал как проверку корректности вычислений.